

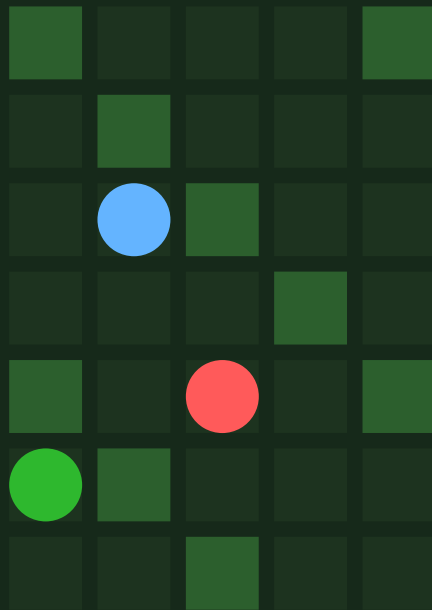


Simulador de Ecossistema

Programação Orientada a Objetos em C++17 com SFML

*Plantas, presas e predadores interagindo em tempo real —
herança, polimorfismo e comportamento emergente.*

[Seu nome · Disciplina · Data]



O projeto em uma tela

Uma cadeia alimentar simulada: plantas crescem e se espalham, presas forrageiam em bando e fogem, predadores caçam com perseguição preditiva. Tudo emerge da interação entre objetos autônomos.



Planta quadrado verde — cresce em clusters



Presas círculo — forrageia e foge em bando



Predador triângulo — caça a presa mais vantajosa

60×60

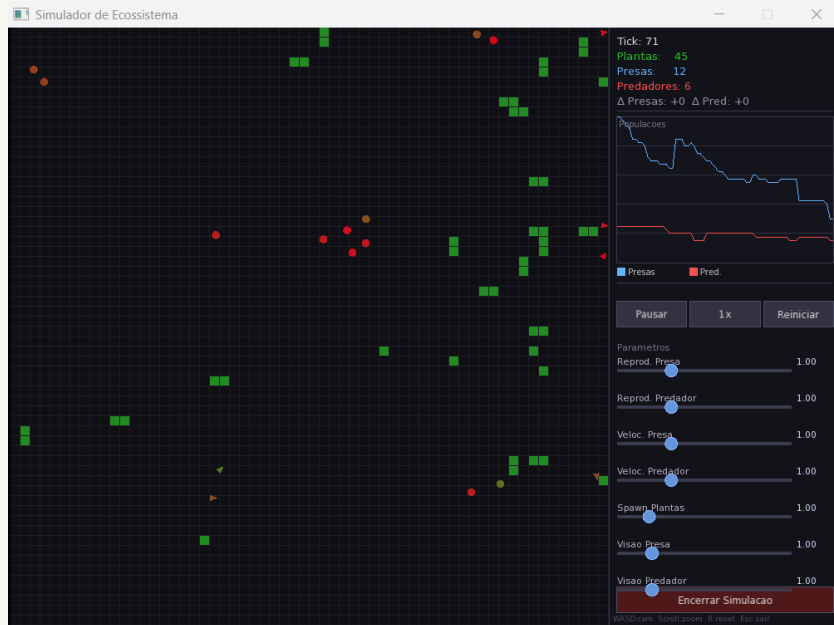
células na grade
espacial

10/s

ticks de simulação (até
8x)

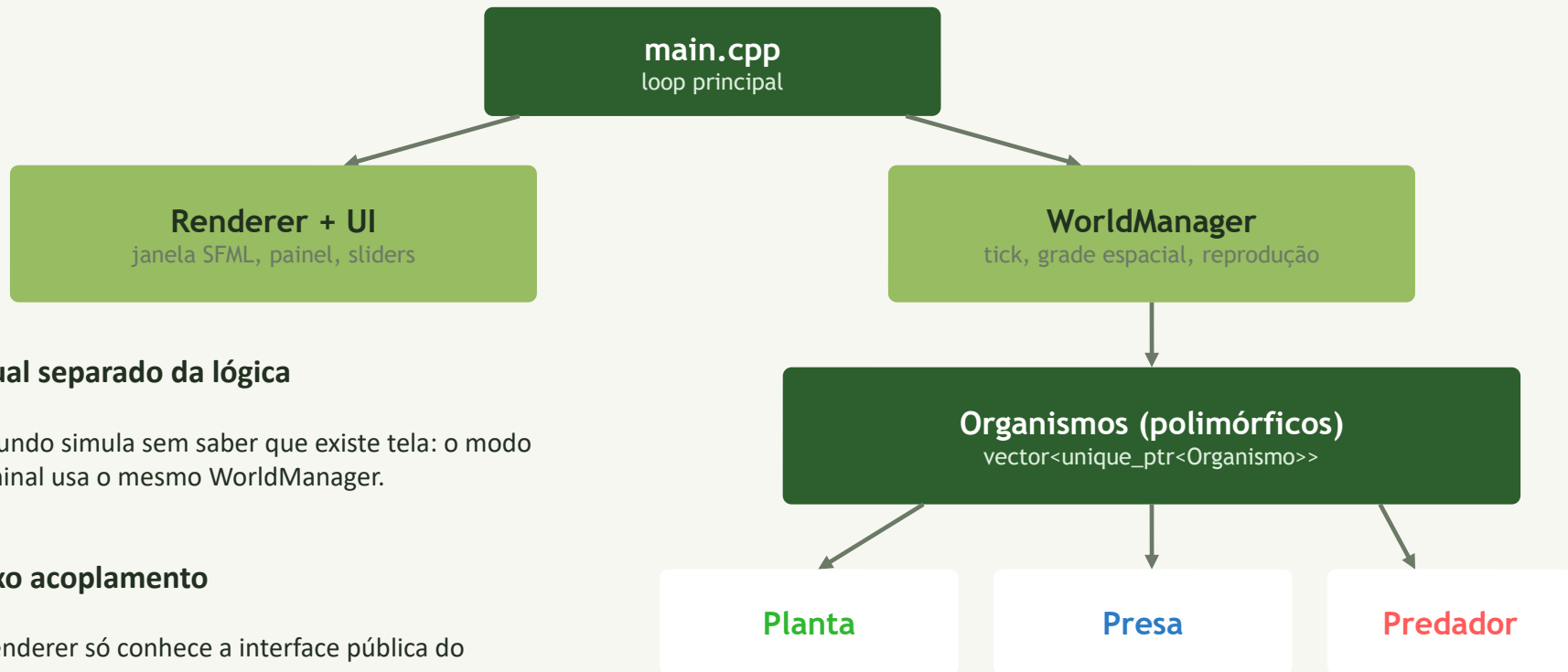
7

parâmetros ajustáveis
ao vivo



A simulação em execução: grid, painel de estatísticas, gráfico de populações e sliders.

Arquitetura em camadas



Visual separado da lógica

O mundo simula sem saber que existe tela: o modo terminal usa o mesmo WorldManager.

Baixo acoplamento

O Renderer só conhece a interface pública do mundo; os organismos não conhecem a UI.

Os quatro pilares da POO no projeto



Abstração

Organismo é uma classe abstrata: define O QUE todo ser vivo faz (agir, mover, desenhar), sem dizer COMO. Cada espécie preenche o contrato.



Herança

Hierarquia em dois níveis: Organismo → Planta e Organismo → Animal → Presa / Predador. Animal concentra o que é comum a quem se move.



Polimorfismo

Um único loop trata todos: org->agir(world) despacha dinamicamente para o comportamento da espécie. Sem if (tipo == ...) na lógica central.



Encapsulamento

Estado interno protegido com invariantes: ninguém altera energia diretamente — consumirEnergia() garante que energia $\leq 0 \Rightarrow$ morte.

Abstração — o contrato de todo ser vivo

src/core/Organismo.hpp

```
class Organismo {
protected:
    float x, y, energia;
    int idade;  bool vivo;
public:
    // metodos virtuais puros: cada
    // especie DEVE implementar
    virtual void agir(WorldManager&) = 0;
    virtual void mover(WorldManager&) = 0;
    virtual void desenhar(...) const = 0;
    virtual TipoOrganismo getTipo() const = 0;
    virtual float getEnergiaParaReproducao() const = 0;

    // destrutor virtual: essencial!
    virtual ~Organismo() = default;
};
```

1

Interface, não implementação

Quem usa um Organismo não precisa saber se é planta ou predador — só que ele sabe agir, mover e se desenhar.

2

= 0 obriga as filhas

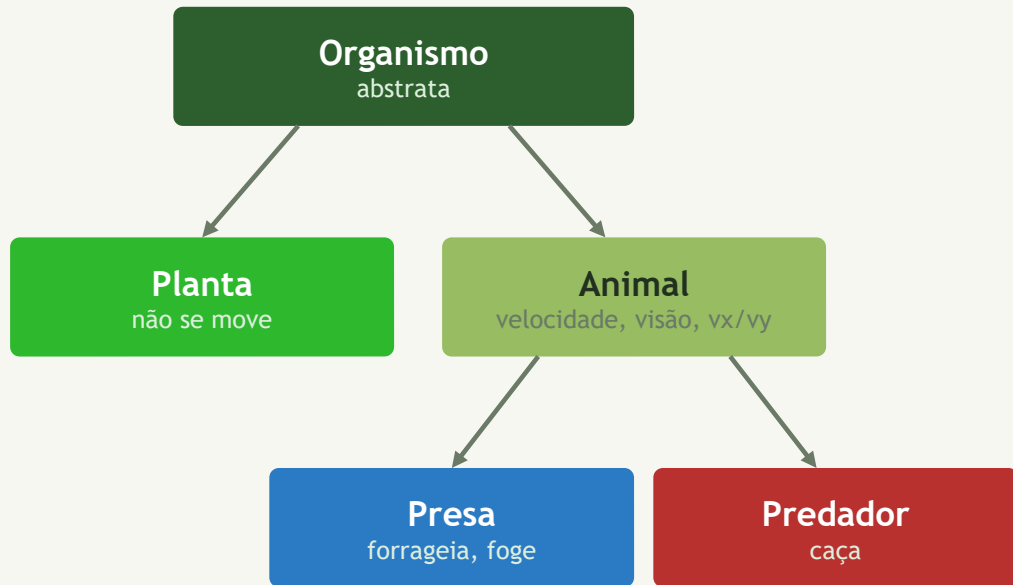
Métodos virtuais puros tornam a classe abstrata: é impossível instanciar um “ser vivo genérico”.

3

Destrutor virtual

Essencial: os objetos são destruídos via ponteiro para a base — sem ele, seria comportamento indefinido.

Herança — especializar sem repetir



Animal é uma base intermediária: concentra física de movimento (aplicarForça), visão e velocidade. Planta herda direto de Organismo — não paga pelo que não usa.

src/entities/Presas.cpp

```
// construtor encadeia a base:
Presas::Presas(float x, float y)
: Animal(x, y,
    ENERGIA_INICIAL,
    COOLDOWN_REPRODUCAO,
    VELOCIDADE, RAO_VISAO) {}

// e Animal encadeia Organismo:
Animal::Animal(...)
: Organismo(x, y, energia,
    cooldown),
    velocidade(velocidade),
    raioVisao(raioVisao) {}
```

Polimorfismo — um loop para todos

src/core/WorldManager.cpp

```
// o mundo nao sabe quem e quem:
for (auto& org : organismos) {
    if (org->estaVivo()) {
        org->agir(*this); // despacho
        org->mover(*this); // dinamico
    }
}
```

A mesma chamada `org->desenhar(...)` produz três formas diferentes — é o que aparece na tela:



`Planta::desenhar` → `RectangleShape`



`Presa::desenhar` → `CircleShape`



`Predador::desenhar` → `ConvexShape` rotacionado

identificação sem `dynamic_cast`

```
// cada classe responde quem e:
TipoOrganismo Presa::getTipo()
    const override {
        return TipoOrganismo::Presa;
}

// e parametros por especie:
float getEnergiaParaReproducao()
    const override {
        return 40.0f; // Presa
    } // Predador retorna 55.0f
```

override pega erros em compilação; a reprodução no `WorldManager` usa esses getters polimórficos sem conhecer as espécies.

Encapsulamento — invariantes protegidas

```
src/core/Organismo.cpp
```

```
void Organismo::consumirEnergia(
    float quantidade) {
    energia -= quantidade;
    if (energia <= 0) {
        energia = 0;
        vivo = false; // invariante
    }
}
```

```
src/core/WorldManager.hpp
```

```
private:
    // ninguem fora ve a grade
    std::vector<std::vector<
        Organismo*>> gradeEspacial;
```

Estado morre junto com a regra

energia e vivo são protected. A ÚNICA porta de entrada é consumirEnergia() — é impossível um organismo ficar com energia negativa e “vivo”.

Estrutura interna trocável

A grade espacial é private: o resto do código usa obterVizinhos(). A otimização de $O(n)$ para $O(1)$ foi feita sem tocar nas espécies.

Leitura sem escrita

Getters const (getX, getEnergia, getOrganismos) expõem o estado para o Renderer sem permitir modificação.

Composição, RAI e o padrão Factory

```
src/core/OrganismoFactory.cpp
```

```
std::unique_ptr<Organismo>
OrganismoFactory::criar(TipoOrganismo t,
                        float x, float y) {
    switch (t) {
        case TipoOrganismo::Planta:
            return std::make_unique<Planta>(x, y);
        case TipoOrganismo::Presas:
            return std::make_unique<Presas>(x, y);
        case TipoOrganismo::Predador:
            return std::make_unique<Predador>(x, y);
    }
}
```

```
posse exclusiva – sem new/delete manuais
```

```
// WorldManager COMPOE os organismos:
std::vector<std::unique_ptr<Organismo>>
    organismos; // destruicao automatica
```

Factory Method

Um único ponto cria qualquer espécie. A reprodução chama `criar(org->getTipo(), x, y)` — funciona para uma espécie nova sem mudar o `WorldManager`.

RAI com `unique_ptr`

O mundo é dono dos organismos: quando um morre e sai do vector, a memória é liberada — zero vazamentos, zero delete.

Composição vs. herança

`WorldManager` TEM organismos (composição); `Presas` É UM `Animal` (herança). Cada relação no mecanismo certo.

Comportamento por máquina de estados

```
src/core/Animal.hpp
```

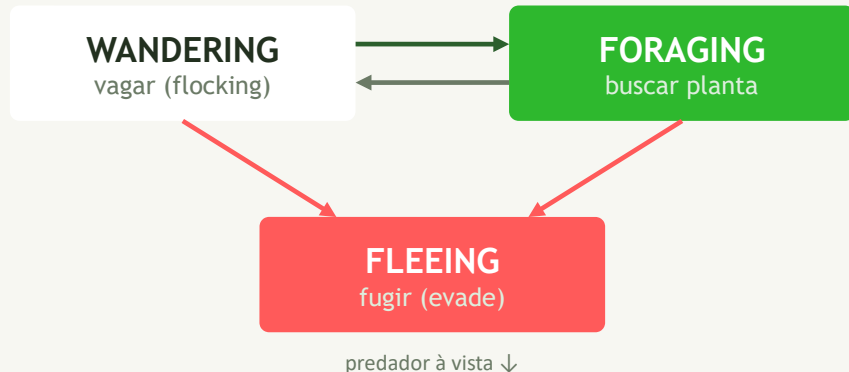
```
enum class EstadoAgente {  
    WANDERING,  
    HUNTING,    // predador  
    FORAGING,   // presa  
    FLEEING     // presa  
};
```

Cada animal decide em agir() e executa em mover(). A transição depende do que ele vê na vizinhança — e fugir custa 1,5x mais energia.

[Espaço para imagem/zoom de presas fugindo em bando — opcional]

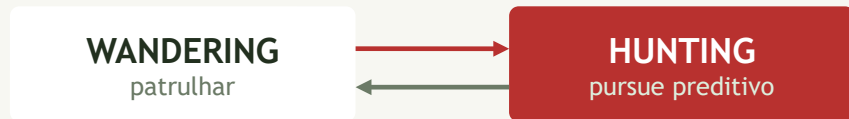
FSM da Presa

viu planta → / ← sem plantas



FSM do Predador

presa no raio de visão → / ← alvo perdido ou comido



Steering → comportamento emergente

seek / flee ir até um alvo / afastar-se dele

pursue / evade antecipa a posição futura do alvo

wander vageio suave com jitter aleatório

flocking separação + alinhamento + coesão

Nenhuma linha de código diz “formem bandos” ou “cresçam em moitas” — isso emerge das regras locais de cada objeto.



Clusters de plantas e o gráfico predador–presa (ciclo de Lotka-Volterra) — tudo emergente.

Engenharia além da POO: desempenho

0,07 ms

por tick com ~700 organismos
(medido com -O2)

$O(1)$

busca de vizinhos via grade espacial
(antes: varredura $O(n)$)

~2100 → 3

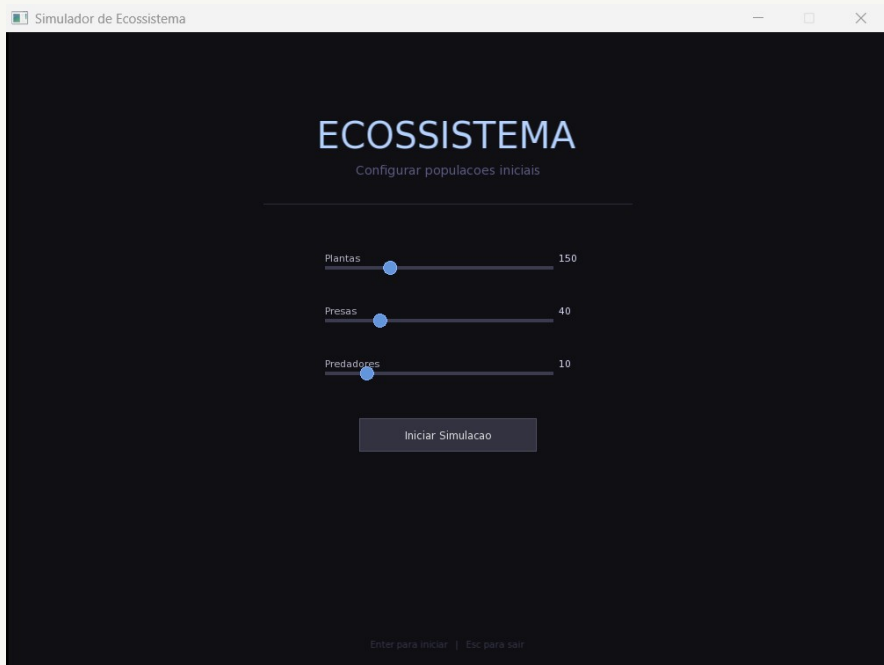
draw calls por frame com batching
em sf::VertexArray

validação de ponteiros $O(1)$ — encapsulada

```
// alvos guardados entre ticks podem morrer;  
// conjunto de vivos valida em  $O(1)$ :  
std::unordered_set<const Organismo*> vivosSet;  
  
bool organismoExisteEVivo(const Organismo* p) {  
    return vivosSet.count(p) && p->estaVivo();  
}
```

Encapsulamento pagou aqui: a troca de $O(n)$ para $O(1)$ aconteceu dentro do WorldManager — nenhuma espécie precisou mudar.

Demonstração



Tela inicial: o usuário escolhe as populações antes de começar.

Controles ao vivo

Espaço

pausa / retoma

F

velocidade 1x → 2x → 4x → 8x

Scroll

zoom no grid

WASD

mover a câmera

Sliders

7 parâmetros em tempo real

Esc

sair

Sugestão: subir “Reprod. Presa” e ver a população de predadores responder no gráfico.



O que a POO comprou neste projeto

Extensibilidade

Uma espécie nova (ex.: necrófago) = uma subclasse nova. O loop do mundo, a fábrica e o renderer já estão prontos para ela.

Manutenção localizada

Mudar a IA da presa não toca o predador; otimizar a grade não toca as espécies. Cada mudança tem um endereço.

Complexidade emergente

Objetos simples + regras locais + polimorfismo = ciclos populacionais que ninguém programou explicitamente.

Obrigado!

[Seu nome · contato / repositório GitHub]